SQLite Cheat Sheet

📌 Installation de SQLite

🪟 Windows

1. Télécharger SQLite :

https://www.sqlite.org/download.html

2. Télécharger :

- `sqlite-tools-win32-x86.zip`

3. Extraire le ZIP dans un dossier, par exemple :

```
C:\sqlite
```

4. Ajouter le dossier au `PATH` Windows.

5. Vérifier l'installation :

```
sqlite3 --version
```

🍎 macOS

SQLite est généralement déjà installé.

Vérifier :

```
sqlite3 --version
```

Si SQLite n'est pas installé :

Avec Homebrew

Installer Homebrew :

Homebrew

Puis :

```
brew install sqlite
```

Vérifier :

```
sqlite3 --version
```

🐧 Linux (Ubuntu/Debian)

Installer SQLite :

```
sudo apt update
sudo apt install sqlite3
```

Vérifier :

```
sqlite3 --version
```

📌 Créer une base de données

```
sqlite3 ma_base.db
```

SQLite crée automatiquement le fichier s'il n'existe pas.

📌 Commandes SQLite Shell

Commande	Description
`.tables`	Liste les tables
`.schema`	Affiche la structure
`.headers on`	Affiche les noms des colonnes
`.mode column`	Affichage en colonnes
`.quit` ou `.exit`	Quitter SQLite

📌 Types SQLite

Type	Description
INTEGER	Entier
REAL	Nombre décimal
TEXT	Texte
BLOB	Données binaires
NULL	Valeur vide

📌 Créer une table

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    email TEXT UNIQUE,
    age INTEGER
);
```

📌 Voir les tables

```
.tables
```

📌 Voir la structure d'une table

```
.schema users
```

📌 Insérer des données

```
INSERT INTO users (nom, email, age)
VALUES ('Rojo', 'rojo@test.com', 30);
```

📌 Lire des données

Toutes les lignes

```
SELECT * FROM users;
```

Colonnes spécifiques

```
SELECT nom, age FROM users;
```

Avec condition

```
SELECT * FROM users
WHERE age > 18;
```

📌 Trier les résultats

Ordre croissant

```
SELECT * FROM users
ORDER BY age ASC;
```

Ordre décroissant

```
SELECT * FROM users
ORDER BY age DESC;
```

📌 Limiter les résultats

```
SELECT * FROM users
LIMIT 5;
```

📌 Modifier des données

```sql
UPDATE users
SET age = 31
WHERE id = 1;
```

📌 Supprimer des données

```sql
DELETE FROM users
WHERE id = 1;
```

📌 Supprimer une table

```sql
DROP TABLE users;
```

📌 Ajouter une colonne

```sql
ALTER TABLE users
ADD COLUMN telephone TEXT;
```

📌 Compter des lignes

```sql
SELECT COUNT(*) FROM users;
```

📌 Valeurs distinctes

```
SELECT DISTINCT age FROM users;
```

📌 Recherche avec LIKE

```
SELECT * FROM users
WHERE nom LIKE '%jo%';
```

Symbole	Signification
`%`	plusieurs caractères
`_`	un seul caractère

📌 Conditions multiples

```
SELECT * FROM users
WHERE age > 18
AND nom LIKE 'R%';
```

📌 IN

```
SELECT * FROM users
WHERE age IN (18, 20, 25);
```

📌 BETWEEN

```
SELECT * FROM users
WHERE age BETWEEN 18 AND 30;
```

📌 NULL

```sql
SELECT * FROM users
WHERE email IS NULL;
```

📌 Clés étrangères

```sql
CREATE TABLE commandes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    montant REAL,

    FOREIGN KEY(user_id)
    REFERENCES users(id)
);
```

📌 Jointures

```sql
SELECT users.nom, commandes.montant
FROM users
JOIN commandes
ON users.id = commandes.user_id;
```

📌 GROUP BY

```sql
SELECT age, COUNT(*)
FROM users
GROUP BY age;
```

📌 HAVING

```sql
SELECT age, COUNT(*)
FROM users
GROUP BY age
HAVING COUNT(*) > 1;
```

📌 Index

```sql
CREATE INDEX idx_users_nom
ON users(nom);
```

📌 Transactions

```sql
BEGIN TRANSACTION;

UPDATE users
SET age = 40
WHERE id = 1;

COMMIT;
```

Rollback :

```sql
ROLLBACK;
```

📌 Import CSV

```
.mode csv
.import users.csv users
```

📌 Export CSV

```
.headers on
.mode csv
.output export.csv

SELECT * FROM users;

.output stdout
```

📌 Sauvegarde

```
sqlite3 ma_base.db ".backup sauvegarde.db"
```

📌 Exécuter un fichier SQL

```
sqlite3 ma_base.db < script.sql
```

📌 Bonnes pratiques

✅ Utiliser `INTEGER PRIMARY KEY AUTOINCREMENT`

✅ Créer des index sur :

- colonnes recherchées
- clés étrangères

✅ Toujours utiliser :

- `WHERE` dans `UPDATE`
- `WHERE` dans `DELETE`

✅ Utiliser des transactions pour les opérations critiques

📌 Outils graphiques utiles

- DB Browser for SQLite
- SQLiteStudio
- DBeaver

📌 Exemple complet

```sql
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    prix REAL,
    stock INTEGER DEFAULT 0
);

INSERT INTO products (nom, prix, stock)
VALUES
('Clavier', 50.0, 10),
('Souris', 25.0, 15);

SELECT * FROM products;

UPDATE products
SET stock = stock - 1
WHERE id = 1;

DELETE FROM products
WHERE stock = 0;
```

📌 Ressources officielles

SQLite Official Website
https://www.sqlite.org/

SQLite Documentation
https://www.sqlite.org/docs.html